

Expressions & Casting

Int vs double: $5/2==2$, $5/2.0==2.5$. Casts change *value* or *type* at that point only.
Char/Int mix: chars are small integers; printing with %c vs %d.

```
1 int a = 2.5;
2 // a=2 (truncation)
3 printf("%d -> %c\n", a, a+'a');
4 double b = a / (char)2.5;
5 // (char)2.5 -> 2, so b=1.0
6 printf("%.2f\n", b);
7 printf("%d\n", ('=' ? 1 : 0));
8 // nonzero char is true
9 int t = (8>0 && 1<2) ? (1==0) : (0==0); // 0 or 1
```

Inc/Dec & Truthiness

```
1 int x=3,y=2;
2 if (x-- > y) printf("%d\n", x);
3 else printf("%d\n", 2*x);
4 if (y) x = !(y<1); else x = !(y>1);
5 printf("%d\n", x);
```

Loop Control

```
1 for (int i=0;i<2;i++){
2     int j=0,a=1;
3     while (j<3){
4         j++;
5         if (j==i) continue;
6         if (j==2) break;
7         a *= 2;
8     }
9     printf("%d:%d\t", i, a);
10 }
```

I/O Basics

printf: %d %u %x %o %c %s %f %e %g %p, width/precision %8d %.2f.
scanf: pass *addresses*; %lf for double; check return count.

Array vs Pointer & sizeof

```
1 void f(char x[], char *y){
2     printf("%zu\n", sizeof x);
3     // pointer size in params
4     x += 3; printf("%c\n", x[-1]);
5     (*y)++;
6     // mutate via pointer
7 }
8 int main(void){
9     char a[6]="cat";
10    printf("%zu\n", sizeof a);
11    // true array size (incl. '\0')
12    f(a, &a[2]);
13 }
```

Pointer Arithmetic & Return

```
1 int* process(int *x){ *x += 1; x = x+2; return x; }
2 int a = 5; int *p = process(&a);
3 // modifies a, moves p
4 int b[4]={7,5,3};
5 p = process(b+1);
6 // pointer math on arrays
```

Double/Char Conversions

```
1 char c = 1.5; // c=1
2 double y='d', z=1.5;
3 c += (int)2.75; // c=1+2=3
4 printf("%.2f\n", c+0.5);
5 printf("%c\n", (char)(y+3*z));
6 printf("%.2f\n", (double)((int)3*z));
```

Memory & calloc

```
1 int *p = (int*)calloc(4, sizeof *p);
2 printf("%zu\n", sizeof p);
3 // size of pointer (e.g., 8 on 64-bit)
4 *p = p[0] + 2;
5 // p[0] initially 0
6 p = p + 1;
7 // pointer rebase
8 printf("%d %d\n", p[-1], p[1]);
9 free(p-1);
10 // free original base if saved
```

Prototypes, Side-Effects, Returns

```
1 int dec(int x,int y){ x=4; printf("%d\n",x);
2     return 2+y; }
3 double inc(double *x,double *y){ *x+=1; *y-=2;
4     return *x+1; }
```

Static vs Globals & Shadowing

```
1 int g;
2 // global default-initialized to 0
3 int calc(void){ static int s=3; s = g + s;
4     return s; }
5 int process(int a){ a=a+a; return a; }
6 // parameter shadows
7 int main(void){
8     printf("%d\n", g);
9     // 0
10    g=0; printf("%d\n", process(++g));
11    g=1;
12    { int g=5;
13        // inner shadow
14        printf("%d\n", g + calc());
15        printf("%d\n", process(++g));
16        g=2;
17        // local change only
18    }
19    printf("%d\n", g + calc());
20 }
```

Enums & 2D Arrays

```
1 typedef enum { mo=4, tu, we, th=9, fi } days;
2 int a[2][3] = {3,5,7, 9,11,13};
3 days b = we; // 6
4 printf("%d\n", a[1][1]); // 11
5 printf("%d %d\n", b, b+1); // 6 7
```

Create Filtered Copy (letters only)

```
1 void create(const char *in, char *out){
2     int j=0;
3     for (int i=0; in[i]; i++){
4         char c=in[i];
5         if ((c>='A'&&c<='Z')||(c>='a'&&c<='z'))
6             out[j++]=c;
7     }
8     out[j]='\0';
9 }
```

Analyze: count symbol & return start ptr

```
1 void analyze(const char *s,
2             int *count,
3             const char **start){
4     int c=0;
5     for (int i=0; s[i]; i++) if (s[i]=='*') c++;
6     *count=c; *start=s;
7 }
```

Next Word Iterator (resettable)

```
1 const char* next(const char *s, int reset){
2     static const char *p;
3     if (reset) p=s;
4     if (!p || !*p) return NULL;
5     while (*p==' ') p++; if (!*p) return NULL;
6     const char *ret=p; while (*p && *p!=' ') p++;
7     return ret;
8 }
```

Modify First Letter to Last Digit of Position

```
1 void modify(char *s){
2     /* assumes s[0] in 'a'..'z' */
3     int pos = (s[0]-'a'+1);
4     // 1..26
5     s[0] = '0' + (pos % 10);
6     // last digit w/o branches
7 }
```

Cut/Resize to Match Another String

```
1 void cut(char *a, const char *b){
2     while (*a && *b){ a++; b++; }
3     if (*a){ *a='\0'; }
4     // shorten a if longer than b
5 }
```

Two-Digit Helpers

```
1 int two_digits(int x){
2     return x>=10 && x<=99; }
3 void swap(int *x){
4     int t=*x; *x = 10*(t%10) + t/10; }
5 void swapif(int *x){
6     if (two_digits(*x)) swap(x); }
7 void process(int *v, int count){
8     for (int i=0;i<count;i++) swapif(&v[i]);
9 }
```

Structs with Embedded/Pointer Fields

```
1 typedef struct { int number; char street[50]; }
2 Address;
3 typedef struct {
4     int pid; char name[50];
5     int *grades; Address address;
6 }
7 Student;
8
9 void print_name(Student s){
10     printf("%s\n", s.name); }
11 void set_pid(Student *s, int pid){
12     s->pid=pid; }
13 void init_grades(Student *s, int n){
14     s->grades = (int*)calloc(n, sizeof *s->grades);
15 }
16 void set_grade(Student s, int idx, int val){
17     s.grades[idx]=val; }
18 char get_street_letter(Student s){
19     return s.address.street[0]; }
```

Random Dice Roll

```
1 typedef struct { int value; } Dice;
2 void roll(Dice *d){ d->value = rand()%6 + 1; }
3 int main(void){
4     srand(time(NULL));
5     Dice d; for (int i=0;i<10;i++){
6         roll(&d); printf("%d ", d.value); }
7 }
```

Swap with First Negative (two styles)

```

1 void swap1(int *a, int n, int *b){
2     for (int i=0;i<n;i++) if (a[i]<0){
3         int t=a[i]; a[i]=*b; *b=t; break; }
4     }
5 int swap2(int *a, int n, int b){
6     for (int i=0;i<n;i++) if (a[i]<0){
7         int t=a[i]; a[i]=b; return t; }
8     return b;
9 }

```

Core API Used

Actions: move(), turn_left(), turn_right(), put_item(), take_item(), turn_off().

Queries: wall_in_front(), wall_to_left(), wall_to_right(), item_present(), bag_empty(), facing_*().

Line Up Items from a Pile Along a Wall

```

1 void place_east(){
2     while(!wall_in_front())
3         move(); put_item();
4     turn_around(); go_to_wall(); turn_around(); }
5 void distribute_once(){
6     take_item(); place_east(); move(); }
7 // adjust per task

```

Count Doors in a Row (north/south openings)

```

1 void step(){ if(!wall_in_front()) move(); }
2 void count_doors(){
3     // place an item at start for each door encountered
4     // check openings with wall_to_left/right per spec;
5     // accumulate with trips
6 }

```

Helper Macros

```

1 void turn_right(){
2     turn_left(); turn_left(); turn_left(); }
3 void turn_around(){ turn_left(); turn_left(); }
4 void go_to_wall(){
5     while(!wall_in_front()) move(); }

```

Return to Start or Required Corner

```

1 void to_start_of_row(){
2     turn_around(); go_to_wall(); turn_around(); }

```